Du notebook à l'application

Tutoriel pas à pas : mettre un modèle de régression linéaire derrière une API (FastAPI) et une interface (Streamlit)

Jeu de données : Advertising (200 observations)
Outils : Python, uv, scikit-learn, FastAPI, Streamlit

Introduction : ce que nous construisons

Un modèle de machine learning entraîné dans un notebook reste inutilisable pour quelqu'un qui ne sait pas programmer. Pour qu'il serve dans la vie réelle, on le place derrière un **service** (une API) et on lui ajoute une **interface** (un formulaire). Ce tutoriel construit ces deux briques, étape par étape.

Le jeu de données Advertising associe trois budgets publicitaires, **TV**, **radio** et **newspaper** (en milliers de dollars), aux **ventes** (*sales*), également en milliers de dollars. Le modèle apprend la relation entre les budgets et les ventes, puis prédit les ventes pour de nouveaux budgets.

Architecture

L'utilisateur remplit un formulaire Streamlit. Streamlit envoie les valeurs à l'API FastAPI sous forme de requête HTTP contenant du JSON. L'API transmet les valeurs au pipeline (standardisation puis modèle) et renvoie la prédiction en JSON. Streamlit l'affiche. **Streamlit ne contient aucun modèle** : il ne fait que dialoguer avec l'API. C'est cette séparation qui illustre une architecture client / serveur.

Schéma du flux

```
Streamlit (formulaire)
   |  requête HTTP POST, corps JSON : {"TV": 100, "radio": 20, "newspaper": 10}
   v
FastAPI  ->  pipeline.predict  (standardisation + modèle)
   |  réponse JSON : {"sales": 11.26}
   v
Streamlit (affichage du résultat)
```

Structure finale du projet

Arborescence

```
demo-regression/
|-- pyproject.toml          dépendances du projet (créé par uv)
|-- uv.lock                 versions exactes installées
|-- .gitignore
|-- .env                    réglages locaux (non versionné)
|-- .env.example            modèle de .env (versionné)
|-- README.md
|-- train_model.py          découpe, entraîne et sauvegarde le pipeline
|-- test_api.py             test de l'API en Python
|-- data/Advertising.csv
|-- models/
|   `-- pipeline.joblib     scaler + régression
|-- api/
|   |-- __init__.py
|   |-- main.py             routes de l'API
|   `-- schemas/
|       |-- __init__.py     ré-exporte les schémas
|       `-- budget.py       classe Budget
`-- app/
    `-- streamlit_app.py
```

Étape 1 : installer uv et créer le projet

uv est un outil moderne qui remplace à la fois *pip* et *venv* : il crée l'environnement virtuel, installe les paquets et mémorise leurs versions. Installez-le une seule fois sur votre machine.

Terminal : installation de uv (une seule fois)

```
# Linux / macOS
curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Terminal : création du projet

```
# Crée le dossier demo-regression avec un pyproject.toml minimal
uv init demo-regression

# On se place dans le dossier : toutes les commandes suivantes partent d'ici
cd demo-regression
```

Explication. La commande `uv init` génère un dossier contenant `pyproject.toml`, le fichier qui décrit le projet et, plus tard, ses dépendances. Rester à la racine du projet est important : les chemins relatifs utilisés dans les scripts (par exemple `models/pipeline.joblib`) sont calculés depuis ce dossier.

Étape 2 : ajouter les dépendances et créer les dossiers

Terminal : dépendances

```
# Ajoute les paquets au projet ET les installe dans l'environnement virtuel
uv add fastapi "uvicorn[standard]" streamlit requests joblib pandas python-dotenv "scikit-learn==1.6.1"
```

Terminal : dossiers

```
# Crée les dossiers du projet
mkdir api app models data
mkdir api/schemas

# Supprime le main.py généré par défaut (inutile ici)
#   Linux / macOS : rm main.py
#   Windows       : del main.py
```

Explication. Chaque paquet a un rôle précis. **fastapi** construit l'API et **uvicorn** est le serveur qui l'exécute. **streamlit** fournit l'interface web. **requests** permet à Streamlit d'appeler l'API. **joblib**, **pandas** et **scikit-learn** servent à charger le modèle et à préparer les données. **python-dotenv** lira le fichier `.env` (étape 7).

Deux détails méritent attention. D'abord, `uvicorn[standard]` : les crochets demandent des dépendances optionnelles, dont *watchfiles*, qui relance l'API automatiquement à chaque modification du code. Les guillemets évitent que certains terminaux interprètent les crochets. Ensuite, `scikit-learn==1.6.1` : un fichier .joblib doit être relu avec la version de scikit-learn qui l'a créé : épingler le numéro garantit que tous les postes, et plus tard le serveur de déploiement, utilisent la même. Pour connaître votre version : `import sklearn; print(sklearn.__version__)`.

Étape 3 : entraîner et sauvegarder le pipeline

Placez `Advertising.csv` dans le dossier `data/`, puis créez `train_model.py` à la racine du projet. Ce script charge les données, les découpe en deux jeux, entraîne un **pipeline** et le sauvegarde dans un seul fichier.

train_model.py

```
import joblib
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# 1. Chargement des données depuis le fichier CSV
df = pd.read_csv("data/Advertising.csv")

# X : les variables explicatives (les 3 budgets)
# L'ORDRE des colonnes est important : il devra être identique dans l'API.
X = df[["TV", "radio", "newspaper"]]

# y : la variable à prédire (les ventes)
y = df["sales"]

# 2. Découpage en deux jeux :
#    - 80 % pour apprendre (train)
#    - 20 % pour évaluer sur des données jamais vues (test)
# random_state=42 fixe le tirage au sort : le découpage est identique à chaque
# exécution, donc les résultats sont reproductibles.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 3. Pipeline : une chaîne d'étapes exécutées dans l'ordre.
#    Chaque étape est un couple (nom, objet).
pipeline = Pipeline([
    ("scaler", StandardScaler()),         # étape 1 : standardiser les budgets
    ("regression", LinearRegression()),   # étape 2 : régression linéaire
])

# 4. Entraînement sur le jeu d'apprentissage UNIQUEMENT.
#    fit apprend la moyenne et l'écart-type (scaler), standardise les données,
#    puis ajuste la régression sur les données standardisées.
pipeline.fit(X_train, y_train)

# 5. Évaluation sur le jeu de test.
#    predict standardise automatiquement avant de prédire.
y_pred = pipeline.predict(X_test)

# R2  : part de la variation des ventes expliquée par le modèle (1 = parfait)
# RMSE: erreur typique, en milliers de $ (racine de l'erreur quadratique moyenne)
# MAE : erreur absolue moyenne, en milliers de $
print(f"R²   : {r2_score(y_test, y_pred):.3f}")
print(f"RMSE : {mean_squared_error(y_test, y_pred) ** 0.5:.3f}")
print(f"MAE  : {mean_absolute_error(y_test, y_pred):.3f}")

# 6. Sauvegarde d'UN SEUL fichier : le pipeline complet (scaler + régression)
joblib.dump(pipeline, "models/pipeline.joblib")

# 7. Contrôle visuel : un coefficient par variable (données standardisées).
#    named_steps permet de récupérer une étape du pipeline par son nom.
print("Coefficients :", pipeline.named_steps["regression"].coef_)
```

Terminal

```
uv run python train_model.py
```

Résultat attendu (valeurs relevées en exécutant ce script)

```
R²   : 0.899
RMSE : 1.782
MAE  : 1.461
Coefficients : [3.76419618 2.79230665 0.05597551]
```

Pourquoi train_test_split ?

Évaluer un modèle sur les données qui ont servi à l'entraîner est trompeur : il peut simplement les « retenir ». On met donc de côté 20 % des lignes (`test_size=0.2`) que le modèle ne voit jamais pendant l'apprentissage, puis on mesure sa qualité sur elles. `random_state=42` fixe le tirage au sort : le découpage est identique à chaque exécution, et les résultats sont reproductibles.

Pourquoi un Pipeline ?

Un `Pipeline` enchaîne plusieurs étapes (ici `StandardScaler` puis `LinearRegression`) dans **un seul objet**. Trois avantages. **Un seul fichier** à sauvegarder et à charger, au lieu de deux (modèle et scaler) qu'il faudrait garder synchronisés. `predict` **standardise automatiquement** avant de prédire : l'API n'a plus à appeler `scaler.transform` elle-même, donc ne peut plus l'oublier. Enfin, **pas de fuite de données** : `fit` calcule la moyenne et l'écart-type sur le jeu d'apprentissage seulement. Standardiser *avant* le découpage laisserait des informations du jeu de test dans le scaler.

Lire les résultats

Le **R^2** d'environ 0,90 signifie que le modèle explique environ 90 % de la variation des ventes sur le jeu de test. Le **RMSE** (1,78) et le **MAE** (1,46) sont des erreurs exprimées en milliers de dollars, soit environ 1 780 $ d'écart typique et 1 460 $ d'erreur absolue moyenne. Les **coefficients** (dans l'ordre TV, radio, newspaper) portent sur des données standardisées, donc comparables entre eux : la TV (3,76) et la radio (2,79) pèsent fortement sur les ventes, les journaux (0,056) presque pas.

À savoir : ce pipeline est entraîné sur 80 % des données. En production, on réentraîne parfois sur la totalité une fois l'évaluation faite ; ici on garde le pipeline évalué, pour que les chiffres affichés correspondent au modèle réellement utilisé.

Étape 4 : le squelette de l'API

Créez un fichier vide `api/__init__.py` : il indique à Python que le dossier `api` est un paquet importable. Puis créez `api/main.py`.

api/main.py (version 1 : squelette)

```
from pathlib import Path

import joblib
from fastapi import FastAPI

# Chemin du dossier models/, calculé à partir de l'emplacement de CE fichier.
# Path(__file__)   : chemin de main.py
# .resolve()       : chemin absolu
# .parent.parent   : on remonte de api/ vers la racine du projet
MODELS_DIR = Path(__file__).resolve().parent.parent / "models"

# Chargement du pipeline UNE SEULE FOIS, au démarrage de l'API.
# (le relire à chaque requête serait inutilement lent)
pipeline = joblib.load(MODELS_DIR / "pipeline.joblib")

# L'objet application : c'est lui que uvicorn va exécuter
app = FastAPI(title="API de prédiction des ventes")


# Le décorateur déclare une route : requête GET sur l'adresse "/"
@app.get("/")
def accueil():
    # FastAPI convertit automatiquement le dictionnaire Python en JSON
    return {"message": "API ok"}
```

Terminal : lancement de l'API

```
# api.main -> fichier api/main.py ; app -> l'objet FastAPI dans ce fichier
# --reload  -> redémarre l'API automatiquement quand le code change
uv run uvicorn api.main:app --reload
```

Explication. Ce squelette fait trois choses : il retrouve le dossier `models/` quel que soit le dossier depuis lequel on lance la commande, il charge le pipeline une seule fois en mémoire, et il déclare une route de test. Ouvrez `http://127.0.0.1:8000` : vous devez voir le message JSON. Ouvrez aussi `http://127.0.0.1:8000/docs` : FastAPI génère une documentation interactive (Swagger) à partir de votre code.

Étape 5 : le schéma de données et la route de prédiction

5a. Le schéma, dans un fichier séparé

Le schéma décrit le JSON que l'API accepte. On le place dans son propre fichier pour garder `main.py` centré sur les routes. Créez `api/schemas/budget.py` :

api/schemas/budget.py

```
from pydantic import BaseModel, Field


class Budget(BaseModel):
    """Corps JSON attendu par /predict (validé automatiquement).
    Mauvais type ou valeur négative -> réponse 422, sans écrire de 'if'."""

    # ge=0 : "greater or equal", la valeur doit être >= 0
    # float : les budgets ont des décimales (230.1) ; un float accepte aussi
    #         les entiers (100 devient 100.0), l'inverse n'est pas vrai.
    TV: float = Field(ge=0, description="Budget TV (k$)")
    radio: float = Field(ge=0, description="Budget radio (k$)")
    newspaper: float = Field(ge=0, description="Budget newspaper (k$)")
```

Créez ensuite `api/schemas/__init__.py`. Ce fichier **ré-exporte** la classe : il permet d'écrire l'import court `from .schemas import Budget` au lieu du chemin complet.

api/schemas/__init__.py

```
# Rend la classe disponible directement depuis le paquet "schemas"
from .budget import Budget
```

Explication. La classe `Budget` est un contrat : elle décrit les champs attendus, leur type et leurs limites. FastAPI s'en sert pour valider chaque requête et pour documenter l'API dans `/docs`. Le choix de `float` tient aux données : un `int` rejetterait `230.1`. Ces budgets sont des variables d'entrée d'un modèle statistique, pas des montants comptables : le type se choisit selon l'usage de la donnée.

5b. La route /predict

On complète `api/main.py`. Voici le fichier **complet** ; les nouveautés sont l'import de `pandas` et de `Budget`, et la route `/predict`.

api/main.py (version finale)

```
from pathlib import Path

import joblib
import pandas as pd
from fastapi import FastAPI

# Import relatif : le point signifie "dans le même paquet que ce fichier" (api/)
from .schemas import Budget

MODELS_DIR = Path(__file__).resolve().parent.parent / "models"

# Chargement unique au démarrage : un seul fichier, le pipeline complet
pipeline = joblib.load(MODELS_DIR / "pipeline.joblib")

app = FastAPI(title="API de prédiction des ventes")


@app.get("/")
def accueil():
    return {"message": "API ok"}


# POST : on ENVOIE des données (le corps JSON) pour obtenir un calcul.
# Le paramètre "budgets: Budget" demande à FastAPI de valider le JSON reçu
# avec le schéma, puis de nous donner un objet Budget prêt à l'emploi.
@app.post("/predict")
def predict(budgets: Budget):
    # 1. Un tableau d'UNE ligne, avec les mêmes colonnes, dans le même ordre
    #    qu'à l'entraînement (TV, radio, newspaper).
    X = pd.DataFrame(
        [[budgets.TV, budgets.radio, budgets.newspaper]],
        columns=["TV", "radio", "newspaper"],
    )

    # 2. Un seul appel : le pipeline standardise (scaler déjà ajusté) PUIS
    #    prédit. predict renvoie un tableau, on prend le premier élément.
    sales = pipeline.predict(X)[0]

    # 3. float(...) convertit le type numpy en float Python (sérialisable en JSON)
    return {"sales": round(float(sales), 2)}
```

Explication. Dans la fonction `predict`, le tableau pandas reprend les **mêmes noms de colonnes** que l'entraînement. Un seul appel, `pipeline.predict(X)`, enchaîne la standardisation puis la prédiction : l'API ne manipule plus de scaler, donc ne risque ni de l'oublier ni d'en ajuster un nouveau. Le pipeline mémorise aussi les noms de colonnes vus à l'entraînement : si le tableau porte d'autres noms ou un autre ordre, scikit-learn lève une `ValueError` au lieu de produire un résultat faux en silence (constaté avec scikit-learn 1.9.1). Ce contrôle n'existe pas avec un simple tableau numpy sans noms : d'où le DataFrame. Enfin, le point de `from .schemas import Budget` exige que l'API soit lancée comme un paquet, avec `uvicorn api.main:app`.

Erreurs fréquentes. `FileNotFoundError` sur `pipeline.joblib` : lancez d'abord `train_model.py`. `ImportError: cannot import name 'Budget'` : le fichier `schemas/__init__.py` est vide, ou le nom de la classe diffère de celui de l'import (`Budget` et `Budgets` sont deux noms distincts).

Étape 6 : tester l'API

Méthode universelle : un script Python

Cette méthode fonctionne à l'identique sous Windows, Linux et macOS, sans problème de guillemets. Créez `test_api.py` à la racine du projet.

test_api.py

```
import requests

# POST avec json= : requests convertit le dictionnaire en JSON
# et ajoute lui-même l'en-tête Content-Type: application/json
reponse = requests.post(
    "http://127.0.0.1:8000/predict",
    json={"TV": 100, "radio": 20, "newspaper": 10},
)

# Code HTTP (200 = succès) puis contenu de la réponse
print(reponse.status_code, reponse.json())
```

Terminal 2 (l'API tourne dans le terminal 1)

```
uv run python test_api.py
# Résultat attendu : 200 {'sales': 11.26}
```

Avec curl

curl est pratique, mais la gestion des guillemets dépend du terminal. Sous **PowerShell**, `curl` est en réalité un alias de `Invoke-WebRequest` et les guillemets du JSON sont mal transmis : utilisez plutôt *cmd*, le script Python, ou la page `/docs`.

Linux / macOS (bash, zsh)

```
# Guillemets simples à l'extérieur : ils protègent les guillemets doubles du JSON
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"TV": 100, "radio": 20, "newspaper": 10}'
```

Windows : invite de commandes (cmd)

```
:: Le ^ prolonge la commande sur la ligne suivante.
:: Guillemets doubles à l'extérieur, \" pour ceux du JSON
curl -X POST http://127.0.0.1:8000/predict ^
  -H "Content-Type: application/json" ^
  -d "{\"TV\": 100, \"radio\": 20, \"newspaper\": 10}"
```

Étape 7 : l'interface Streamlit

7a. Le formulaire, avec l'adresse de l'API en variable d'environnement

Créez `app/streamlit_app.py`. Streamlit exécute ce fichier de haut en bas à chaque interaction ; chaque appel `st.xxx` ajoute un élément à la page.

app/streamlit_app.py

```
import os

import requests
import streamlit as st

# Adresse du SERVEUR de l'API, sans la route.
# os.environ.get("API_URL", défaut) cherche une variable d'environnement
# nommée API_URL ; si elle n'existe pas, on utilise la valeur par défaut
# (l'API lancée sur votre propre machine).
API_URL = os.environ.get("API_URL", "http://127.0.0.1:8000")

# On ajoute la route ici : elle fait partie du code, pas de la configuration
PREDICT_URL = f"{API_URL}/predict"

st.title("Prédiction des ventes")
st.write("Veuillez saisir les budgets publicitaires")

# st.form regroupe les champs : la page ne se relance qu'au clic sur le bouton,
# et non à chaque modification d'un champ.
with st.form("formulaire"):
    # min_value interdit les négatifs, value = valeur initiale, step = pas
    tv = st.number_input("TV", min_value=0.0, value=100.0, step=10.0)
    radio = st.number_input("Radio", min_value=0.0, value=20.0, step=1.0)
    newspaper = st.number_input("Newspaper", min_value=0.0, value=10.0, step=1.0)

    # Le bouton d'envoi renvoie True au moment du clic
    submit = st.form_submit_button("Calculer les ventes")

# Ce bloc ne s'exécute qu'après un clic sur le bouton
if submit:
    # Mêmes noms de clés que ceux du schéma Budget de l'API
    donnees = {"TV": tv, "radio": radio, "newspaper": newspaper}
    try:
        # Appel de l'API, avec un délai maximal de 5 secondes
        response = requests.post(PREDICT_URL, json=donnees, timeout=5)

        # Lève une exception si le code HTTP est une erreur (4xx, 5xx)
        response.raise_for_status()

        # On lit la valeur "sales" dans le JSON de la réponse
        sales = response.json()["sales"]
        st.success(f"Ventes prévues : {sales} k$")
    except requests.exceptions.RequestException:
        # API éteinte, délai dépassé, erreur HTTP... : message clair
        st.error("Impossible de joindre l'API.")
```

Terminal : lancement (l'API tourne déjà dans un autre terminal)

```
uv run streamlit run app/streamlit_app.py
# Le navigateur s'ouvre sur http://localhost:8501
```

Explication. Le formulaire collecte les trois budgets. Au clic, `requests.post` envoie les valeurs à l'API : c'est exactement le même appel que dans `test_api.py`. Le bloc `try / except` remplace une longue trace d'erreur par un message compréhensible si l'API est éteinte. Avec les valeurs par

défaut (100, 20 et 10), vous devez obtenir **11,26 k$**. Il faut **deux terminaux** ouverts en même temps : l'un pour l'API, l'autre pour l'interface.

Comprendre la ligne API_URL

L'adresse de l'API change selon l'endroit où l'application tourne : `http://127.0.0.1:8000` sur votre machine, une adresse publique une fois déployée. Plutôt que de modifier le code à chaque fois, on la lit dans une **variable d'environnement**, un réglage fourni au programme depuis l'extérieur.

Les deux « API_URL » de la ligne

```
API_URL = os.environ.get("API_URL", "http://127.0.0.1:8000")
#  ^                        ^
#  variable Python          NOM de la variable d'environnement (du texte)
#  (à gauche du =)          il doit être identique, au caractère près, à celui
#                           défini dans le terminal ou dans le fichier .env
```

Si le nom ne correspond pas (par exemple `API_url` dans le `.env`), Python ne trouve rien et utilise **silencieusement** la valeur par défaut. C'est le piège le plus courant. Sans valeur par défaut, `os.environ.get("API_URL")` renverrait `None`, l'adresse deviendrait `None/predict` et l'application afficherait « Impossible de joindre l'API ». Pourquoi l'adresse sans `/predict` ? Parce que le serveur change selon l'environnement alors que la route fait partie du code ; et sans slash final, sinon l'URL deviendrait `...com//predict`.

7b. Charger un fichier .env avec load_dotenv()

Définir la variable dans le terminal est possible, mais à refaire à chaque session. Un fichier `.env` la conserve. **Python ne lit pas ce fichier tout seul** : sans appel explicite, il est ignoré et la valeur par défaut s'applique en silence. `load_dotenv()` ouvre le `.env` et ajoute son contenu à `os.environ`, là où `os.environ.get` va chercher.

Le chemin de la valeur

```
.env  --load_dotenv()-->  os.environ  --os.environ.get(...)-->  variable Python API_URL
```

Modifiez le début de `app/streamlit_app.py`. L'appel doit venir **avant** la lecture de `API_URL`, sinon le `.env` est chargé trop tard.

app/streamlit_app.py (début du fichier)

```
import os

import requests
import streamlit as st
from dotenv import load_dotenv

# Lit le fichier .env (s'il existe) et ajoute son contenu aux variables
# d'environnement. Une variable déjà définie dans le terminal n'est pas écrasée.
load_dotenv()

API_URL = os.environ.get("API_URL", "http://127.0.0.1:8000")
PREDICT_URL = f"{API_URL}/predict"

# ... le reste du fichier est inchangé
```

.env (à la racine du projet)

```
API_URL=http://127.0.0.1:8000
```

Règles de syntaxe : pas d'espace autour du `=`, pas de `/predict` ni de slash final, et un nom identique à celui du code. **Ordre de priorité :** variable du terminal, puis `.env`, puis valeur par défaut du code.

Vérification. Remplacez le port par `9999` dans le `.env`, arrêtez Streamlit (Ctrl+C) et relancez-le **une seule fois** : le message « Impossible de joindre l'API. » doit apparaître. Remettez `8000` ensuite. Si rien ne change, deux causes fréquentes : `load_dotenv()` est absent ou placé après la lecture de la variable, ou une ancienne instance de Streamlit tourne encore (elle occupe le port 8501 et la nouvelle démarre sur 8502 sans bruit).

Pour déboguer. Ajoutez temporairement `st.write(API_URL)` : la page affiche la valeur réellement lue. N'oubliez pas de la retirer.

Étape 8 : préparer le dépôt Git

.gitignore

```
# Environnement virtuel : lourd et propre à votre machine
# (recréé par "uv sync" à partir de pyproject.toml et uv.lock)
.venv/

# Caches générés automatiquement par Python
__pycache__/
*.pyc

# Secrets et réglages locaux : ne jamais envoyer sur GitHub
.env
.streamlit/secrets.toml
```

.env.example (versionné : sert de modèle)

```
API_URL=http://127.0.0.1:8000
```

À versionner : `pyproject.toml` et `uv.lock` (ils fixent les versions exactes, dont scikit-learn 1.6.1), `.env.example` et `models/pipeline.joblib`, sans lequel l'application ne peut pas prédire (il est léger ici). Le `.env` lui-même reste local : chacun copie le modèle et l'adapte. Un autre poste reconstruit alors l'environnement complet avec les commandes ci-dessous.

Terminal : cloner et relancer sur une autre machine

```
git clone <url-du-dépôt>
cd demo-regression

# Recrée l'environnement à l'identique à partir de uv.lock
uv sync

# Crée son propre .env à partir du modèle
cp .env.example .env          # Windows : copy .env.example .env

# Terminal 1 : l'API
uv run uvicorn api.main:app --reload

# Terminal 2 : l'interface
uv run streamlit run app/streamlit_app.py
```

Dépannage

Symptôme	Cause probable	Solution
{"detail":"Not Found"} sur /predict	Fichier non sauvegardé ou API non rechargée	Sauvegarder api/main.py, vérifier /docs, relancer
unmatched close brace (curl)	Guillemets mal transmis par PowerShell	Utiliser cmd ou test_api.py
Erreur 422	JSON invalide ou budget négatif	Vérifier les noms TV, radio, newspaper et les valeurs
Impossible de joindre l'API	API non lancée, ou API_URL à None / erronée	Démarrer uvicorn ; vérifier le défaut et le .env
ImportError: cannot import name 'Budget'	schemas/__init__.py vide, ou nom de classe différent	Y mettre from .budget import Budget
ImportError: attempted relative import	main.py lancé directement	Utiliser uvicorn api.main:app
Le .env semble ignoré	load_dotenv() absent ou appelé trop tard	L'appeler avant os.environ.get
Aucun changement ne se voit	Ancienne instance de Streamlit encore active	Tout arrêter (Ctrl+C), relancer une seule fois
FileNotFoundError sur pipeline.joblib	Pipeline pas encore créé	Lancer train_model.py
ValueError: feature names should match	Colonnes du DataFrame différentes de l'entraînement	Utiliser [TV, radio, newspaper] dans cet ordre
Avertissement scikit-learn au chargement	Versions différentes	Épingler la version utilisée à l'entraînement

Pour aller plus loin

Schéma de réponse. Ajouter une classe `PredictionResponse` pour documenter aussi la sortie de l'API dans `/docs`. **Déploiement.** Streamlit Community Cloud n'héberge que l'interface : l'API doit être déployée ailleurs, puis la variable `API_URL` doit pointer vers son adresse publique. **Tests.** Ajouter des tests automatisés (pytest) sur la route `/predict`. **Aller plus loin avec le pipeline.** La validation croisée (`cross_val_score`) et la recherche d'hyperparamètres (`GridSearchCV`) s'appliquent directement au pipeline, sans fuite de données.